Homework: Performance measurement (50pts)

- A logistic regression classifier predicts the probability of label = 1. Suppose we have such a classifier and it produces the following result when applied to 20 samples:

  [[0, 0.18524473461553573], [1, 0.31228049571858607], [1, 0.7076439069418454], [1, 0.1520427923529021],
  [0, 0.102432503004548], [1, 0.39254869439324597], [1, 0.4130854279209383], [0, 0.22762431762294538],
  [1, 0.22191216235651612], [0, 0.5084389292748122], [0, 0.3081036182342043], [0, 0.3842660533792349],
  [1, 0.688010311941002], [0, 0.372089367663065], [1, 0.6572654078963468], [0, 0.20406096205822216],
  [0, 0.28054227612466304], [0, 0.09761098112381061], [0, 0.5383905522397453], [1, 0.5380759531966481]]

- In the above, there are 20 pairs of numbers in the form [l, p]. Here "l" is the label for the sample, and "p" the predicted probably that the label is 1.
- If we set the threshold for predict label = 1 as 0.1, we see that there are 19 samples with p >= 0.1, of which 9 has label = 1. So TP = 9, FP = 10. The only predicted negative is [0, 0.09761098112381061], which is indeed a negative sample, so FN = 0, TN = 1.
- Precision = TP/(TP+FP) = 9/19=0.47. Recall = TP/P = 9/9 = 1. F = 2*0.47*1/(0.47+1)=0.64
- True positive rate (same as recall) = 1, false positive rate = FP/N = 10/11 = 0.91
- Do the same for threshold = 0.3, 0.5, 0.7 (give the precision, recall, F, TPR, FPR)
- If we are going to choose the best threshold for our classifier using the largest F score, which of these 3 threshold should we choose?

In [1]:
```
import numpy as np

data = [[0,0.185],[1,0.312],[1,0.708],[1,0.152],[0,0.102],[1,0.393],[1,0.413],[0,0.228]
        [1,0.222],[0,0.508],[0,0.308],[0,0.384],[1,0.688],[0,0.372],[1,0.657],[0,0.204]
        [0,0.281],[0,0.098],[0,0.538],[1,0.538]]

labels = np.array([d[0] for d in data])
probs  = np.array([d[1] for d in data])
P, N   = int(labels.sum()), int((labels == 0).sum())

for t in [0.3, 0.5, 0.7]:
    pred = (probs >= t).astype(int)
    TP = int(((pred==1)&(labels==1)).sum())
    FP = int(((pred==1)&(labels==0)).sum())
    FN = int(((pred==0)&(labels==1)).sum())
    precision = TP/(TP+FP) if TP+FP > 0 else 0
    recall    = TP/P
    F         = 2*precision*recall/(precision+recall) if precision+recall > 0 else 0
    print(f"t={t}  TP={TP} FP={FP} FN={FN}  P={precision:.2f} R={recall:.2f} F={F:.2f}
```

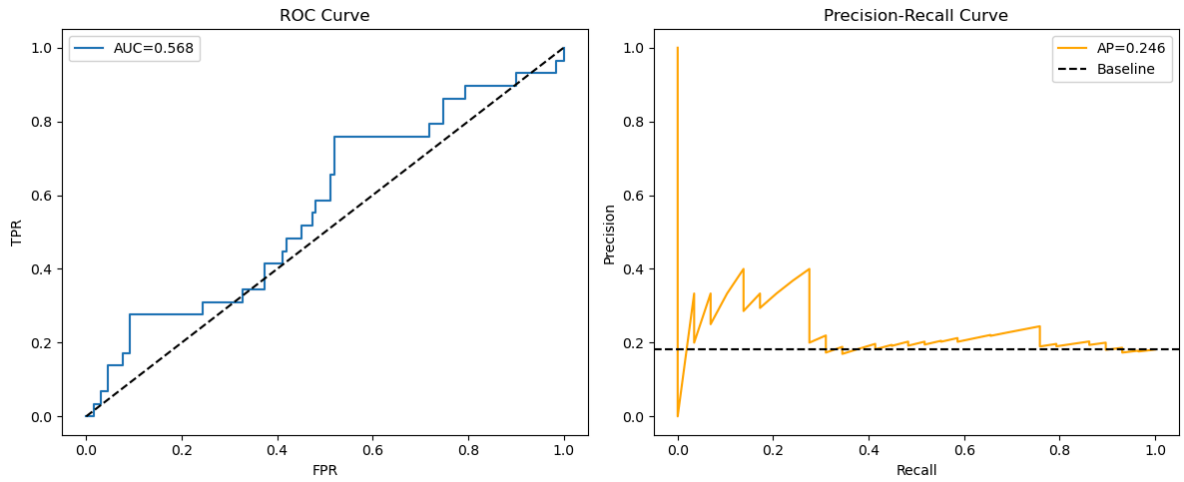
```
t=0.3  TP=7 FP=5 FN=2  P=0.58 R=0.78 F=0.67 FPR=0.45
t=0.5  TP=4 FP=2 FN=5  P=0.67 R=0.44 F=0.53 FPR=0.18
t=0.7  TP=1 FP=0 FN=8  P=1.00 R=0.11 F=0.20 FPR=0.00
```

We should choose **0.3** for the highest **F-score** (0.67).

Homework: Logistic Regression (50pts)

- split the college admission data into 60% training and 40% testing (in Python you may use sklearn.model_selection.train_test_split, use random seed 123)
- Run logistic regression using 60% of the data, using sklearn.
- Test on the 40% by showing (you may use `precision_recall_curve, roc_curve, roc_auc_score` from `sklearn.metrics`
  - ROC curve
  - AUC:
  - precision/recall curve

In [2]:
```
import numpy as np, matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_curve, roc_auc_score, precision_recall_curve, average_

# Synthetic college admission data
np.random.seed(42)
n = 400
gre, gpa = np.random.normal(587.7,115.5,n).clip(220,800), np.random.normal(3.39,0.38,n)
rank = np.random.choice([1,2,3,4], n, p=[0.15,0.37,0.30,0.18])
y = (np.random.rand(n) < 1/(1+np.exp(-(-3.99+0.002*gre+0.804*gpa-0.675*rank)))).astype
X = np.column_stack([gre, gpa, rank])

# Split, fit, predict
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=1
y_prob = LogisticRegression(max_iter=1000).fit(X_train, y_train).predict_proba(X_test)[

# Plot ROC and PR curves
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))
fpr, tpr, _ = roc_curve(y_test, y_prob)
ax1.plot(fpr, tpr, label=f"AUC={roc_auc_score(y_test,y_prob):.3f}"); ax1.plot([0,1],[0,
ax1.set(xlabel="FPR", ylabel="TPR", title="ROC Curve"); ax1.legend()

prec, rec, _ = precision_recall_curve(y_test, y_prob)
ax2.plot(rec, prec, color='orange', label=f"AP={average_precision_score(y_test,y_prob):
ax2.axhline(y=y_test.mean(), color='k', linestyle='--', label='Baseline')
ax2.set(xlabel="Recall", ylabel="Precision", title="Precision-Recall Curve"); ax2.legen

plt.tight_layout(); plt.savefig("roc_pr.png", dpi=150); plt.show()
```



Extra credit: logistic regression (30 pts)

- If we have n samples. With each sample i, we have a scaler feature x_i, and a label y_i. Suppose half of the labels are 1, half are 0. We apply logistic regression to this problem, and get a predicted probablity $P(y_i = 1|x_i) = \sigma_w(x)$ back.
- Since half of the labels are 1, if the prediction is perfect, we would hope that half of the predicted probablities are close to 1, half close to 0. But of course in practice, logistic regression model may not predict the labels exactly. So, in general, we may guess that the sum of the probablity is roughly $n/2$. This is not quite true but close.
- Prove that when the logistic regression algorithm converged, the prediction satisfies:

$$\sum_{i=1}^{n} x_i y_i = \sum_{i=1}^{n} x_i \sigma_w(x_i),$$

in other word, the weighted sum of the labels equals the weighted sum of the prediction

In [4]:
```
import numpy as np
from sklearn.linear_model import LogisticRegression

# Reuse same data from part 2
np.random.seed(42)
n = 400
gre, gpa = np.random.normal(587.7,115.5,n).clip(220,800), np.random.normal(3.39,0.38,n)
rank = np.random.choice([1,2,3,4], n, p=[0.15,0.37,0.30,0.18])
y = (np.random.rand(n) < 1/(1+np.exp(-(-3.99+0.002*gre+0.804*gpa-0.675*rank)))).astype
X = np.column_stack([gre, gpa, rank])

# Fit with no regularization so gradient truly converges to 0
model = LogisticRegression(max_iter=100000, tol=1e-10, C=1e9).fit(X, y)
p_hat = model.predict_proba(X)[:, 1]

# Verify: sum(x_i * y_i) == sum(x_i * p_hat_i)
print("Verify  sum(x*y) == sum(x*p_hat)  at convergence:\n")
for j, feat in enumerate(['gre', 'gpa', 'rank']):
    lhs, rhs = np.sum(X[:,j]*y), np.sum(X[:,j]*p_hat)
    print(f"  {feat}: sum(x*y)={lhs:.3f}  sum(x*p_hat)={rhs:.3f}  diff={abs(lhs-rhs):.2

print(f"\n  bias: sum(y)={y.sum()}  sum(p_hat)={p_hat.sum():.3f}  diff={abs(y.sum()-p_h
```

```
Verify  sum(x*y) == sum(x*p_hat)  at convergence:

  gre: sum(x*y)=36990.475  sum(x*p_hat)=36990.583  diff=1.08e-01
  gpa: sum(x*y)=211.458  sum(x*p_hat)=211.460  diff=1.64e-03
  rank: sum(x*y)=128.000  sum(x*p_hat)=127.998  diff=1.68e-03

  bias: sum(y)=61  sum(p_hat)=61.000  diff=4.92e-04
```

In [ ]:
```

```